

Esercizio 1 (Pekko): Sistema d'Allarme Domestico Basato su Attori

1. Analisi del problema

L'obiettivo dell'esercizio è progettare un **sistema d'allarme domestico concorrente**, capace di gestire:

- molteplici sensori di tipo diverso (movimento, finestre, porte)
- zone indipendenti
- stati operativi distinti (disarmed, exitDelay, armed, entryDelay, alarm)
- ritardi temporali (EXIT DELAY, ENTRY DELAY)
- input asincroni provenienti da attori diversi (sensori, keypad, sirena)
- attivazione e disattivazione della sirena

Il problema è intrinsecamente **concorrente**: i sensori possono generare eventi in qualunque momento, il controller deve mantenere uno stato coerente anche in presenza di input asincroni, e i timer introducono ulteriori eventi che competono con i messaggi degli attori.

Aspetti concorrenti rilevanti

1. **Eventi asincroni e indipendenti:** Ogni sensore è un attore autonomo che può inviare notifiche di trigger in qualunque momento. Il controller deve decidere se ignorare o gestire l'evento in base allo stato corrente e alle zone attive.
2. **Isolamento degli attori:** Il modello ad attori garantisce che ogni attore processi i messaggi in modo sequenziale, eliminando race condition interne. Questo è particolarmente utile per il controller, che rappresenta una macchina a stati.
3. **Timer concorrenti** I timer EXIT DELAY e ENTRY DELAY generano messaggi asincroni che competono con gli eventi dei sensori. La loro gestione deve essere integrata nella logica degli stati.
4. **Macchina a stati:** La logica del sistema è naturalmente modellabile come una macchina a stati. Ogni stato definisce quali messaggi sono validi e quali transizioni sono possibili. Questo approccio riduce drasticamente la complessità concorrente.
5. **Configurazione dinamica delle zone:** L'utente può decidere quali zone attivare al momento dell'inserimento del PIN. Questo introduce un ulteriore livello di concorrenza: la scelta delle zone influenza il comportamento dei sensori in tempo reale.

2. Strategia adottata

2.1 Architettura modulare a più attori

Il sistema è composto da attori indipendenti che comunicano tra loro:

- **AlarmControllerActor:** È il cuore del sistema. Implementa la macchina a stati, gestisce i timer, mantiene le zone attive e decide come reagire agli eventi dei sensori.
- **KeypadActor** Rappresenta l'interfaccia utente. Riceve il PIN e le zone da attivare e le inoltra al controller.
- **SensorActor** Ogni sensore è un attore autonomo che invia eventi Trigger al controller.
- **SirenActor** Gestisce l'attivazione e la disattivazione della sirena.

2.2 Macchina a stati esplicita

Il controller è implementato come una macchina a stati, dove ogni stato è una funzione che restituisce un nuovo Behavior. Gli stati sono:

- **DISARMED** Il sistema è inattivo. I sensori vengono ignorati. L'utente può inserire il PIN e scegliere le zone da attivare.
- **EXIT DELAY** Ritardo che permette all'utente di uscire prima che il sistema diventi operativo. I sensori vengono ignorati.
- **ARMED** Il sistema è attivo. Solo i sensori appartenenti alle zone armate possono generare un ENTRY DELAY.
- **ENTRY DELAY** Ritardo che permette all'utente di disarmare il sistema prima che scatti l'allarme.
- **ALARM** La sirena è attiva. Il sistema ignora tutto tranne il PIN corretto.

2.3 Gestione concorrente dei timer

I timer EXIT DELAY ed ENTRY DELAY sono implementati tramite TimerScheduler. Ogni timer genera un messaggio asincrono (ExitDelayTimeout, EntryDelayTimeout) che viene gestito dallo stato corrente. Questo permette di modellare in modo naturale il comportamento temporale del sistema senza introdurre attese bloccanti.

3. Comportamento del sistema

L'utente può attivare una combinazione qualsiasi di zone, le zone supportate (facilmente ampliabili nel caso) sono: zona giorno (living), zona notte(sleeping), perimetro (perimeter) e garage. Il sistema presenta la simulazione di varie tipologie di sensori: su porte, su finestre e sensori di movimento (interni ed esterni).

3.1 Armamento

L'utente inserisce tramite il tastierino il pin e le zone che si vogliono attivare, a seguito di ciò il controller:

1. valida il PIN
2. attiva le zone richieste

3. avvia l'EXIT DELAY
4. passa allo stato ARMED

3.2 Intrusione

Se un sensore appartenente a una zona attiva invia un evento il sistema entra in "ENTRY DELAY" e se il PIN non viene inserito in tempo si passa allo stato "ALARM".

3.3 Allarme

In stato "ALARM" la sirena è attiva e il sistema ignora tutto tranne il PIN corretto, se viene inserito il PIN corretto il sistema passa in stato "DISARMED" e la sirena viene spenta.

4. Test e scenari

Sono stati implementati vari test relativi a scenari possibili (consultabili dal main del programma), ogni test cerca di simulare una situazione reale in cui il sistema d'allarme è chiamato a funzionare correttamente, i vari scenari ipotizzati sono i seguenti:

- userBackHomeWithFullArmedSystem() => Il Sistema è armato, l'utente torna a casa, vengono triggerati un paio di sensori ma l'utente inserisce il codice corretto prima dello scadere del timer => Il sistema passa in stato "DISARMED"
- intrusionRevealed() => Il sistema è armato completamente, un intruso cerca di entrare il che provoca il trigger di vari sensori, questa volta il pin non viene inserito e l'allarme suona, dopo un po' di tempo l'utente spegne la sirena inserendo il pin corretto.
- nightModeScenario() => Il sistema è in modalità notte, tutti i sensori sono attivi tranne quelli nella zona notte il che permette all'utente di muoversi liberamente e tenere le finestre aperte nella zona notte, senza provocare l'allarme. Alla mattina seguente l'utente triggera il sensore di movimento della zona giorno ma inserisce il pin in tempo e porta il sistema in modalità "DISARMED".
- onlyPerimeterScenario() => Il sistema è in modalità solo perimetro, sono attivi solo i sensori di movimento relativi per esempio a 3 videocamere esterne sul perimetro, all'interno della casa possono avvenire movimenti ed aperture di porte e finestre, solo i sensori sul perimetro provocano un cambio di stato.
- someoneTryingToGuessPin() => Avviene un'intrusione mentre non è presente nessuno in casa, l'intruso cerca di indovinare il pin ma sbaglia, l'allarme continua a suonare senza fermarsi ed il sistema rimane in stato "ALLARM".

5. Conclusioni

L'esercizio ha permesso di progettare un sistema concorrente realistico utilizzando il modello ad attori. La combinazione di: architettura modulare, macchina a stati e isolamento degli attori ha permesso di ottenere un sistema robusto, estensibile e privo di race condition.

Il risultato finale è un impianto d'allarme completo, configurabile e testabile, che rispecchia fedelmente (penso) il comportamento di un sistema reale.

Esercizio 2 (Go): Torneo concorrente “Pari o Dispari”

1. Analisi del problema

L'obiettivo dell'esercizio è progettare un sistema concorrente in Go che simuli un torneo a eliminazione diretta tra N giocatori. Ogni match è una sfida di “pari o dispari” tra due giocatori:

1. in ogni match se un giocatore ha ruolo “pari”, l'altro ha ruolo “dispari” e viceversa
2. entrambi generano un numero casuale
3. la somma determina il vincitore

Il torneo procede a round successivi finché rimane un solo giocatore.

Aspetti concorrenti rilevanti:

1. **Esecuzione parallela dei match:** In ogni round, tutte le partite devono essere eseguite in parallelo tramite goroutine. Questo introduce concorrenza reale, ogni match è indipendente e può terminare in tempi diversi.
2. **Comunicazione tramite canali:** I giocatori non sono funzioni passive: ciascuno è rappresentato da una goroutine che:
 - riceve una richiesta tramite un canale
 - genera un numero casuale
 - risponde tramite un canale di ritorno
3. **Sincronizzazione dei risultati:** Il coordinatore deve:
 - raccogliere i log dei match in ordine deterministico
 - raccogliere i vincitori in modo sincronizzato
 - avanzare al round successivo solo quando tutte le partite sono terminate

Questo richiede un uso corretto dei canali per evitare race condition o deadlock.

4. **Randomness indipendente per ogni giocatore:** Ogni giocatore ha il proprio generatore di numeri casuali. Questo evita correlazioni indesiderate tra i valori generati.
5. **Gestione strutturata del torneo:** Il coordinatore deve:
 - mescolare i giocatori a ogni round (random shuffle)
 - avviare i match in parallelo
 - raccogliere i risultati
 - costruire la lista dei giocatori che avanzano

Tutto questo avviene senza alcuna forma di lock esplicito: la sincronizzazione è affidata interamente ai canali.

2. Strategia adottata

2.1 Coordinatore centralizzato con comunicazione tramite canali

Il coordinatore (runTournament) gestisce l'intero torneo:

- crea la lista dei giocatori attivi
- mescola l'ordine a ogni round
- avvia i match in parallelo
- raccoglie i log e i risultati tramite due canali distinti
- garantisce che i log vengano stampati in ordine deterministico

L'uso di due canali separati (logs e results) permette di mantenere l'ordine dei log.

2.2 Parallelizzazione dei match tramite goroutine

Ogni match viene eseguito in una goroutine separata: questa goroutine riceve due giocatori, invia loro una richiesta e attende le risposte che determinano il vincitore, infine invia log e risultato al coordinatore.

2.3 Struttura a round successivi

Il torneo procede in round, quando un round finisce viene aggiornato l'array contenente i vari id dei giocatori ancora in gioco, il processo continua finché non rimane un solo giocatore (ovvero finché l'array non contiene un unico elemento).

2.4 Avvio del torneo

L'avvio del torneo è molto semplice e viene gestito dal main che si occupa del tutto

1. crea N giocatori
2. avvia il torneo
3. attende la stampa del vincitore

E' fondamentale sottolineare che runTournament (la funzione chiamata dal main alla fine) NON è una goroutine, e questo è il comportamento logicamente corretto: il main deve "fermarsi" su quella funzione non eseguirla in parallelo, sennò il main terminerebbe prima della fine del torneo.

3. Conclusioni

L'esercizio dimostra come Go permetta di modellare sistemi concorrenti in modo semplice ed efficace grazie a:

- goroutine leggere
- comunicazione tramite canali
- sincronizzazione naturale tramite attesa sui canali

Il risultato è un torneo completamente concorrente, riproducibile e privo di race condition, che sfrutta pienamente il modello su cui Go è basato.